

ProductionManagementAI — 完了記録とダッシュボードのクエリ — データベース設計書（テーブル定義書）

003_DB — 要件 REQ-028～REQ-039（work-items/WI-004/brief.md）、基本設計 003_BD（docs/en/010_basic-design/003/003_BD_ダッシュボード.md）。003_DD と 003_DD-API（docs/en/020_detailed-design/003/）、および 001_DD への REQ-033 の修正を実装する。

この文書は 001_DB が定義した 1 つのテーブルを変更する： production_orders に列 1 つとチェック制約 2 つ、部分インデックス 2 つ、デモ用の履歴を追加する。001_DB と 002_DB が products、production_orders、production_order_number_counters について定めていることはすべてそのまま有効であり、繰り返さない。

物理命名は 000_DB～002_DB に従う： snake_case、接尾辞 _utc を付けた timestamptz の UTC タイムスタンプ、EF Core の pk_ / fk_ / ix_ / ck_ 形式の制約名。

文書管理（改版履歴）

項目	値
文書 ID	003_DB
システム名	ProductionManagementAI
作業項目	WI-004
データベース	production_management_ai、スキーマ public
DBMS	PostgreSQL 17
文字コード	UTF-8
作成者	Claude（ThanhTN の依頼による）
作成日	2026-09-22
最終更新者	Claude（ThanhTN の依頼による）
最終更新日	2026-09-28

No	対象	作業項目	変更内容	変更日	変更者
1	全体	WI-004	新規作成（WI-004 DEC-003、DEC-004、DEC-013～DEC-015）	2026-09-22	Claude（ThanhTN の依頼による）
2	全体	WI-005	003_DB に改名（RFC 0010）。英語版・日本語版 PDF を追加（RFC 0008）	2026-09-23	Claude（ThanhTN の依頼による）

No	対象	作業項目	変更内容	変更日	変更者
3	全体	—	ファイル名に画面の日本語名を使用（RFC 0011）	2026-09-24	Claude（ThanhTN の依頼による）
4	全体	—	テーブル定義テンプレートに合わせて再構成：文書管理の項目、ライフサイクル付きテーブル一覧、追加列を完全な列定義形式で記載、ソート順と作成方法付きのインデックス一覧、トリガー一覧、データベースロールの節、マイグレーション表。003_DD を作成済みに更新	2026-09-28	Claude（ThanhTN の依頼による）

テーブル一覧

テーブルの作成・削除はない。ダッシュボードが扱うテーブル（すべて 001_DB で定義）：

No	スキーマ	物理名	論理名	概要	PK 以外のインデックス	トリガー	作成日	スキーマ変更日	削除日	利用状況
1	public	products	製品（001_DB）	製品参照データ。003_DB での変更：なし — 読み取りのみ。製品ラベルのために結合する	有	無	2026-09-18	—	—	使用中
2	public	production_orders	製造指示（001_DB）	製造指示 1 件につき 1 行。003_DB での変更：新しい列 completed_at_utc、チェック制約 2 つ、部分インデックス 2 つ、バックフィル、デモ用の履歴	有	無	2026-09-18	2026-09-22 （completed_at_utc、チェック制約 2 つ）	—	使用中
3	public	production_order_number_counters	製造指示番号採番（001_DB）	年ごとの製造指示番号の連番。003_DB での変更：投入年のカウンターを新しいデモ用製造指示の後まで進める	無	無	2026-09-18	—	—	使用中

ER 図とリレーション

001_DB から変更なし。ダッシュボードがたどるリレーションは 1 つ：

テーブル	関連テーブル	リレーション（1:1 / 1:N / N:N）	FK 列
products	production_orders	1:N	production_orders.product_id

テーブル定義

製造指示（production_orders） — 追加列

テーブルのヘッダーとその他の列はすべて 001_DB にある。追加列（番号は 001_DB の最後の列の続き）：

No	項目名	物理名	データ型	桁数	NOT NULL	既定値	PK	UK	FK	検証	使用	概要説明	区分値	備考
12	CompletedAtUtc	completed_at_utc	timestampz	—	no	なし				アプリケーションが設定し、リクエストからは受け付けない。ck_..._completed_at_matches_status、ck_..._completed_at_not_before_created	○	Completed に なった日時（REQ-033、DEC-003）	—	製造指示を InProgress から Completed にする保存と同じ保存で、その保存の UTC 時刻 — updated_at_utc に書くのと同じ値 — を設定する。その後は変更しない。その他のステータスでは NULL。既存の完了済み製造指示はバックフィルする（マイグレーションの影響を参照）

- **利用パターン（パターン）**：該当なし — 1 種類のレコードのみ。
- **CSV 入出力（DL対象 / UL更新）**：該当なし — CSV 入出力はない。
- **初期データ**：下の「デモ用初期データ」を参照。

履歴テーブルではなく列にする理由：Completed には InProgress からしか到達できず、終端である（001_BD M-02）。したがって製造指示は高々 1 回しか完了せず、1 つのタイムスタンプで事実全体を保持できる。DEC-003 にトレードオフを記録している。

ダッシュボードが読む列

物理名	用途
status	ステータスタイル（REQ-029）。「アクティブ」= Draft または InProgress
due_date	納期超過・納期間近のグループ（D-01、D-02）、負荷の区分（D-03）、納期遵守の比較（D-07）
quantity	負荷と上位製品の数量（D-03、D-04）、完了数量（D-05、D-06）
product_id	上位製品のグループ化（D-04）。ラベルのための products との結合
created_at_utc	リードタイム（D-09）
completed_at_utc	すべての納入指標（D-05～D-09）
id、order_number	一覧表示する最大 20 件の製造指示（D-01、D-02）。タイプブレーカー

notes、updated_at_utc、order_year、order_seq、xmin は読まない。

制約

制約	種別 (PK/FK/UNIQUE/CHECK/EXCLUDE)	テーブル列	参 照 先	ルール
ck_production_orders_completed_at_matches_status	CHECK	production_orders.status、 completed_at_utc	—	(status = 'Completed') = (completed_at_utc IS NOT NULL) — 完了した製造指示は必ず完了日時を持ち、それ以外の製造指示は持たない (REQ-033 の不合格基準)
ck_production_orders_completed_at_not_before_created	CHECK	production_orders.completed_at_utc、 created_at_utc	—	completed_at_utc IS NULL OR completed_at_utc >= created_at_utc — 何が行を書き込んでも、リードタイムが負になることはない (REQ-037 の不合格基準)

001_DB がアプリケーションに任せるルール（納期 ≥ 今日、遷移グラフ）と違い、この 2 つは時計に関係なく常に成り立つので、データベースで強制できる。これはアプリケーションのルールの代わりではなく、その下の安全網である：値を設定するのはエンティティ（001_DD モジュール 1、修正版）であり、設定し忘れたコード経路があれば、納入の数値から製造指示が黙って抜け落ちるのではなく、保存がはっきり失敗する。

遷移ルール自体（InProgress → Completed のみ、Completed は終端）は 001_DB の決定どおりアプリケーションで強制する。Completed は終端なので、1 つ目の制約は、正当な遷移で completed_at_utc が消えることがないことも意味する。

アクセスパターン

ダッシュボードは1 回の読み込みで 7 つの文（Q1～Q6。Q3 は注目グループごとに 1 回）を発行し、1 つのスナップショットを見るよう、すべてを 1 つの読み取り専用 REPEATABLE READ トランザクション内で実行する（DEC-015。トランザクションと同時実行制御を参照）。日付に依存する値はすべて、アプリケーションが工場の時計から計算してパラメーターとして渡す。SQL がタイムスタンプを工場現地の日付に変換する必要がある箇所では、工場のタイムゾーン名自体もパラメーター（@zone、PlantOptions.TimeZone から）として渡す。クライアントからの入力かどの文にも届くことはない — エンドポイントにはパラメーターがない（003_BD §5）。

パラメーター（すべてスナップショットごとに IPlantClock から 1 回計算。T = 工場の今日、W = T の週の月曜日）：

パラメーター	値	使用箇所
@today	T（日付）	Q2～Q4
@soonEnd	T + 7（日付）	Q3（納期間近）
@week0	W(T)（日付）	Q2（負荷の区分）
@trendStartUtc	工場現地の日 W(T) − 77 の開始時刻（UTC の瞬間）	Q5、Q6 の下限
@weekStartUtc	工場現地の日 W(T) の開始時刻（UTC）	Q5（今週の完了）
@monthStartUtc	T の月の 1 日（工場現地）の開始時刻（UTC）	Q5（今月の完了）

パラメーター	値	使用箇所
@windowStartUtc	工場現地の日 T - 29 の開始時刻 (UTC)	Q5 (納期遵守、リードタイム)
@endUtc	工場現地の日 T + 1 の開始時刻 (UTC)	Q5、Q6 の上限
@zone	Asia/Tokyo (設定値)	Q5、Q6 の工場日付への変換

@trendStartUtc は常に最も早い下限である：推移は少なくとも 77 日遡るが、月と 30 日の期間は最大でも 30 日しか遡らない。範囲述語は completed_at_utc 自体に対して UTC の瞬間で書くので、下のインデックスが使える。工場日付への変換はグループ化と納期遵守の比較にだけ現れ、既に絞り込んだ行に対して実行される。

Q1 — ステータス別件数 (REQ-029)

```
SELECT status, count(*) FROM production_orders GROUP BY status;
```

Q2 — 負荷の区分 (D-03、REQ-031)、アクティブな製造指示のみ

```
SELECT CASE
    WHEN due_date < @today          THEN -1          -- 納期超過
    WHEN due_date >= @week0 + 56    THEN  8          -- それ以降
    ELSE (due_date - @week0) / 7    -- 週 0..7
END AS bucket,
count(*), sum(quantity)
FROM production_orders
WHERE status IN ('Draft', 'InProgress')
GROUP BY bucket;
```

due_date - @week0 は日数の整数なので、整数除算により T から W(T) + 55 までの各日付がちょうど 1 つの週 0~7 に入る。行のない区分はアプリケーションが 0 で埋める (REQ-031)。納期超過区分の件数は D-01 の合計でもあり、Q3 が独立に読む — どちらも同じスナップショットからなので一致する。

Q3 — 注目グループ (D-01、D-02、REQ-030)、範囲を変えて 2 回実行

```

SELECT o.id, o.order_number, o.quantity, o.due_date, o.status, p.sku, p.name,
       count(*) OVER () AS total
FROM production_orders o
JOIN products p ON p.id = o.product_id
WHERE o.status IN ('Draft', 'InProgress')
      AND o.due_date < @today                -- 納期超過
      -- または: AND o.due_date BETWEEN @today AND @soonEnd -- 納期間近
ORDER BY o.due_date, o.order_number
LIMIT 10;

```

`count(*) OVER ()` は `LIMIT` の前に評価されるので、1 つの文で最初の 10 行とグループの合計の両方を返す。グループが空なら行を返さず、合計は 0 になる。(以下では Q3a、Q3b と表記する。)

Q4 — 上位製品 (D-04、REQ-032)

```

SELECT p.id, p.sku, p.name, sum(o.quantity) AS open_quantity, count(*) AS active_orders
FROM production_orders o
JOIN products p ON p.id = o.product_id
WHERE o.status IN ('Draft', 'InProgress')
GROUP BY p.id, p.sku, p.name
ORDER BY open_quantity DESC, p.sku
LIMIT 10;

```

Q5 — 納入の数値 (D-05、D-06、D-07、D-09。REQ-034、REQ-035、REQ-037)、1 行

```

SELECT
    count(*)      FILTER (WHERE completed_at_utc >= @weekStartUtc) AS week_count,
    sum(quantity) FILTER (WHERE completed_at_utc >= @weekStartUtc) AS week_quantity,
    count(*)      FILTER (WHERE completed_at_utc >= @monthStartUtc) AS month_count,
    sum(quantity) FILTER (WHERE completed_at_utc >= @monthStartUtc) AS month_quantity,
    count(*)      FILTER (WHERE completed_at_utc >= @windowStartUtc) AS window_count,      -- y
    count(*)      FILTER (WHERE completed_at_utc >= @windowStartUtc
                                AND (completed_at_utc AT TIME ZONE @zone)::date <= due_date)
                                AS window_on_time,      -- x
    avg(extract(epoch FROM completed_at_utc - created_at_utc) / 86400.0)
        FILTER (WHERE completed_at_utc >= @windowStartUtc) AS window_lead_days
FROM production_orders
WHERE completed_at_utc >= @trendStartUtc
    AND completed_at_utc < @endUtc;

```

行がないときの `sum` と `avg` は `NULL` を返す。アプリケーションは `NULL` の合計を 0 に、`NULL` の平均 (`window_count = 0` のとき) を「—」に対応付ける (003_BD M-13、M-14)。丸めと表示はアプリケーションが行うので、データベースは丸めない平均を返す。

Q6 — 完了の推移 (D-08、REQ-036)

```

SELECT date_trunc('week', completed_at_utc AT TIME ZONE @zone)::date AS week_start, count(*)
FROM production_orders
WHERE completed_at_utc >= @trendStartUtc
    AND completed_at_utc < @endUtc
GROUP BY week_start;

```

PostgreSQL の `date_trunc('week', ...)` は ISO 週の月曜日に切り捨てるので、ちょうど `W(d)` になる (DEC-008)。アプリケーションは結果を想定される 12 の月曜日 `W(T) - 77, ..., W(T)` に並べ、欠けた週を 0 で埋める。

`@endUtc` で Q5 と Q6 を上限で区切るので、スナップショットの今日より後の完了タイムスタンプが数値に入ることはない。本番ではありえないが、データベースの `now()` とは独立に工場の時計を固定するテストでは意味がある。

アクセスパターン	頻度	対応するもの
Q1 ステータス別件数	ダッシュボードの読み込みごとに 1 回	<code>production_orders</code> の順次スキャン — 性能を参照
Q2、Q4 アクティブな製造指示の集計	読み込みごとに各 1 回	<code>ix_production_orders_active_due_date</code> (アクティブ集合のビットマップスキャン)

アクセスパターン	頻度	対応するもの
Q3a 納期超過、Q3b 納期間近：納期順の最初の 10 件	読み込みごとに各 1 回	インデックス順の <code>ix_production_orders_active_due_date</code>
Q5、Q6 最近の完了	読み込みごとに各 1 回	<code>ix_production_orders_completed_at_utc</code> の範囲スキャン

インデックス一覧

001_DB と 002_DB の既存インデックスは変更しない。003_DB で新規（どちらも稼働中の `production_orders` テーブルに `AddProductionOrderDashboardIndexes` が作成）：

No	テーブル	インデックス名	方式	一意	列とソート順	部分インデックス条件 / INCLUDE	CONCURRENTLY で作成	作成日	削除日	根拠（アクセスパターン）
1	<code>production_orders</code>	<code>ix_production_orders_active_due_date</code>	btree	no	<code>due_date ASC, order_number ASC</code>	<code>WHERE status IN ('Draft', 'InProgress')</code>	yes — 稼働中のテーブル	2026-09-22	—	Q2～Q4 はアクティブな製造指示だけを読む。アクティブな製造指示は未完了の負荷であり、 <code>Completed</code> と <code>Cancelled</code> が増え続ける一方ではぼ一定なので、時間とともにアクティブ集合はテーブルのごく一部になる。部分インデックスにより、これらのクエリはテーブルの履歴ではなく未完了の負荷に比例する。 <code>(due_date, order_number)</code> の順序は Q3 の <code>ORDER BY</code> そのものなので、各注目グループは順に読まれ、10 行で止まる。002_DB の全体インデックス <code>ix_production_orders_due_date_order_number</code> でも Q3 に対応できるが、最初の納期超過のアクティブな製造指示に達するまでに、古い納期の完了・取消の製造指示をすべてたどってしまう
2	<code>production_orders</code>	<code>ix_production_orders_completed_at_utc</code>	btree	no	<code>completed_at_utc ASC</code>	<code>WHERE completed_at_utc IS NOT NULL</code>	yes — 稼働中のテーブル	2026-09-22	—	Q5 と Q6 は、これまでに記録されたすべての完了のうち直近約 12 週間の完了を読む。このインデックスの範囲スキャンはその期間だけを読む。部分インデックス条件により、未完了の行 — さもなければすべて <code>NULL</code> のエントリになる — を除外する

意図的に追加しなかったインデックス

候補	追加しない理由
<code>status</code> （Q1 用）	Q1 はすべての行のステータスを必要とするので、インデックスが役立つのはインデックスオンリースキャンの場合だけであり、一日中更新されるテーブルでは可視性マップのためにそれが当てにならない。性能で示した規模では、テーブルのスキャンは安価である。約 100 万行を超えたら見直す。その場合の答えはインデックスではなく、ステータス別件数の保持である
<code>(product_id) WHERE active</code> （Q4 用）	Q4 は、どのインデックスで見つけてもアクティブ集合全体を集計する。製品によるグループ化はその集合に対するハッシュ集約である。アクティブの部分インデックスが既に読む行を限定している

候補	追加しない理由
(completed_at_utc) INCLUDE (quantity, due_date, created_at_utc) のカバリングインデックス	Q5/Q6 をインデックスオンリーにできるが、最大でも約 12 週間分の完了のヒープ取得を節約するために、一日中更新されるテーブルのインデックスを広げることになる。この件数では割に合わない

トリガー一覧

なし — トリガーはない。completed_at_utc はトリガーではなくアプリケーションが設定し、設定し忘れたコード経路はチェック制約が検出する。

トランザクションと同時実行制御

操作	トランザクション範囲	同時実行制御
ダッシュボードのスナップショット（Q1～Q6、FN-017）	1 つの明示的なトランザクション BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY ... COMMIT （DEC-015）	PostgreSQL の REPEATABLE READ は、トランザクション内のすべての文に、最初の文の時点で取得した同じスナップショットを与える。したがって 7 つの文はすべて production_orders の一貫した 1 つの状態を見て、003_BD の相互チェック — タイルの合計が総数と一致し、負荷のバーの合計が下書き + 進行中と一致する — が成り立つ。読み取り専用トランザクションはこのレベルでシリアライズ失敗を起こさないで、再試行の経路はない。ACCESS SHARE 以上のロックを取らず、書き込みを妨げない
製造指示の完了（SCR-001 の保存、FN-021）	001_DD-FN UpdateAsync の既存の単一 SaveChangesAsync	変更なし：xmin による楽観的同時実行チェック（WI-002 DEC-010、DEC-014）。completed_at_utc は同じ UPDATE の列が 1 つ増えるだけなので、完了日時とステータス変更はともにコミットされるか、どちらもされない

トランザクションではなく CTE を使った 1 つの文にしない理由：一貫性は同じだが、形の異なる 7 つの結果（グループ化した件数、2 つの行リスト、ランキング 1 行、週ごとの系列）を 1 つの結果セットにまとめることになり、EF Core は無理をしないとマッピングできない。読み取り専用のスナップショットトランザクションなら、各クエリを読みやすく、単独でテストできるまま保てる。

デモ用初期データ

DEC-004（ユーザー）は、納入のウィジェットにデータが入るよう、投入済みのデモ用製造指示にもっともらしい完了日時を与えることを求めている。WI-003 の初期データだけではそれは不可能である。80 件のうち Completed は 16 件しかなく、投入済みの製造指示はすべて初期データの実行より最大 49 日前に作成されている。完了は作成より前にはなりえない。したがってその 16 件では、推移の 12 週のうちの最後の 7 週を、週約 2 件でしか埋められない。そこで DEC-013（ユーザー）は両方を行う：既存の完了済み製造指示 16 件の日付を変更し、過去の完了済み製造指示 40 件を追加する。DEC-014 は、最後の 2 週のバーと「それ以降」のバーを埋めるのに十分先の納期を持つアクティブな製造指示を 4 件追加する。WI-003 の初期データ（納期は最大 40 日先）では確実には届かない。

002_DB（WI-003 DEC-011）と同じく、すべての日付はマイグレーションの実行時点に対する相対値である。その他はすべて定数：ID、製品、数量、日数のオフセット。

日付を変更する既存の完了済み製造指示（16 件）

投入行の seq 6、12、18、24、30、31、37、43、49、55、61、67、68、73、74、80。WI-003 の初期データの ID 接頭辞とステータス Completed で特定する。製品・数量・納期は変更しないので、画面 B の納期フィルターと納期超過マーカは影響を受けない。マイグレーションは completed_at_utc = now() - c 日、updated_at_utc = completed_at_utc（完了の保存が製造指示の最後の更新）、created_at_utc = completed_at_utc - 1 日を設定する。

seq	納期（実行日からの日数、変更なし）	c 日前に完了	リードタイム / 日	作成は何日前	納期遵守
74	+0	0	9	9	遵守
43	+5	1	12	13	遵守
24	−6	2	15	17	遅延
49	−7	3	11	14	遅延
80	−10	4	18	22	遅延
18	+10	6	8	14	遵守
37	+11	8	14	22	遵守
12	+16	10	21	31	遵守
73	+15	12	10	22	遵守
31	+17	14	16	30	遵守
67	+21	17	13	30	遵守
6	+22	20	19	39	遵守
61	+27	23	9	32	遵守
55	+33	26	22	48	遵守
30	+38	29	12	41	遵守
68	+40	33	17	50	遵守

納期よりかなり前に完了した製造指示はもっともらしく、納期遵守である（003_BD D-07）。

新しい過去の完了済み製造指示（40 件）

すべて Completed、備考なし。それぞれ `completed_at_utc = now() - c 日`、`created_at_utc = completed_at_utc - 1 日`、`updated_at_utc = completed_at_utc`、`due_date = 完了の工場現地日付 + δ 日`（δ が負 = 遅れて完了）。

#	製品	数量	c 日前に完了	リードタイム / 日	完了に対する納期 δ	納期遵守
1	P-1001	320	0	11	+3	遵守
2	P-1002	540	2	14	−2	遅延
3	P-1003	150	3	9	+4	遵守
4	P-1004	880	5	16	+1	遵守
5	P-1005	95	7	12	+0	遵守
6	P-1006	1,200	9	20	−3	遅延

#	製品	数量	c 日前に完了	リードタイム / 日	完了に対する納期 δ	納期遵守
7	P-1007	430	11	8	+5	遵守
8	P-1008	260	13	15	+2	遵守
9	P-1009	710	15	10	-1	遅延
10	P-1010	55	16	18	+6	遵守
11	P-1011	1,900	19	13	+3	遵守
12	P-1012	340	21	9	+1	遵守
13	P-1013	620	24	22	-4	遅延
14	P-1014	180	25	11	+2	遵守
15	P-1015	2,500	27	14	+7	遵守
16	P-1016	75	30	17	+0	遵守
17	P-1017	960	32	12	-2	遅延
18	P-1018	410	35	19	+4	遵守
19	P-1019	130	37	10	+1	遵守
20	P-1020	1,450	40	24	-5	遅延
21	P-1021	290	42	13	+3	遵守
22	P-1022	820	45	16	+2	遵守
23	P-1023	65	47	11	-1	遅延
24	P-1024	1,100	50	20	+5	遵守
25	P-1025	370	53	14	+1	遵守
26	P-1026	2,200	56	9	-3	遅延
27	P-1027	145	58	18	+2	遵守
28	P-1028	690	61	12	+4	遵守
29	P-1029	310	64	21	-2	遅延
30	P-1030	1,750	66	15	+3	遵守
31	P-1001	85	69	10	+1	遵守
32	P-1002	560	71	17	-6	遅延
33	P-1003	240	74	13	+2	遵守
34	P-1004	1,300	76	22	+5	遵守

#	製品	数量	c 日前に完了	リードタイム / 日	完了に対する納期 δ	納期遵守
35	P-1005	420	78	11	+0	遵守
36	P-1006	95	80	16	-3	遅延
37	P-1007	780	82	14	+2	遵守
38	P-1008	210	83	19	+4	遵守
39	P-1009	1,600	85	12	-1	遅延
40	P-1010	330	88	20	+3	遵守

新しい納期の遠いアクティブな製造指示（4 件、DEC-014）

#	製品	数量	ステータス	納期（実行日からの日数）	作成は何日前	実行日のバー
41	P-1011	600	Draft	+49	5	週 7（実行日の曜日によらず）
42	P-1017	1,800	Draft	+56	8	それ以降
43	P-1024	450	InProgress	+63	12	それ以降
44	P-1003	950	Draft	+42	3	週 6（実行日の曜日によらず）

`updated_at_utc = created_at_utc`、`completed_at_utc` は `NULL`。週 k は $W(T) + 7k \dots W(T) + 7k + 6$ 、つまり実行日の曜日 $wd = 0$ （月）... 6（日）に対して $T + 7k - wd \dots T + 7k + 6 - wd$ である。したがって $T + 42$ は常に週 6、 $T + 49$ は週 7 に入り、 $T + 56$ 以降は常に「それ以降」になる。製造指示 #44 が必要なのは、月曜日が火曜日に実行すると WI-003 の初期データでは週 6 が空になるためである。

初期データが生む数値

上の行を、各曜日の実行と、月の 1 日・15 日・28 日の実行について、実行日にダッシュボードを見るときとしてシミュレーションして確認した：

数値	結果
完了の推移（D-08）	どの曜日に実行しても、12 週すべてに少なくとも 2 件の完了がある。週約 3 件から、直近の週の 7～10 件へと増える
納期遵守、直近 30 日（D-07）	30 件中 23 件 — 77%、どの曜日に実行しても同じ
平均リードタイム、直近 30 日（D-09）	13.7 日、30 件に基づく
今週の完了（D-05）	月曜日の実行で 2 件、日曜日の実行で 10 件まで増える — 空になることはない
今月の完了（D-06）	月の 1 日で 2 件、月末近くで最大 29 件
ステータスタイル	124 件： Draft 35、InProgress 25、Completed 56、Cancelled 8
負荷（D-03）	納期超過 15。実行日には、どの曜日に実行しても週 0～7 と「それ以降」のすべてのバーが 0 でない。バーの合計は 60（35 + 25）
納期超過／納期間近（D-01、D-02）	実行日に納期超過 15 件（10 件を表示し「15 件中 10 件を表示」）、納期間近 8 件

数値	結果
上位製品（D-04）	27 製品に未完了数量があるので、10 件の一覧は埋まる

WI-003 の納期超過マーカーと同様に、初期データの実行後に日が経つと数値はずれていく。Compose のボリュームを作り直すと、新しい日付で初期データが再実行される。

保護、製造指示番号、適用範囲

- 日付変更は、WI-003 の初期データの ID 接頭辞を持ち、ステータスがまだ Completed の行だけを対象にする。WI-003 の初期データが実行されていないデータベースでは該当行がなく、何もしない。
- 挿入は、WI-003 の投入行が存在し（デモ用データベースである印）、かつ新しい 44 の ID がまだ存在しないときだけ実行する。再適用しても何も起きず、実際の製造指示のデータベースにデモ用の履歴が入ることはない。
- ID は固有の接頭辞 0197e4a0-0000-7000-8002-... を持つ UUIDv7 形の固定数で、WI-003 の ...-8001-... と区別されるため、それぞれの初期データを単独で削除できる。
- 製造指示番号は固定しない。WI-003 の初期データの後にユーザーが既に製造指示を作成している（00081 以降を使っている）可能性があるので、新しい行は投入年のカウンターの次の 44 の連番を使う。その後、同じマイグレーションでカウンターを最後の番号まで進め、画面 A がその後に続くようにする。API で最初に作成した製造指示が 00081 であることを検証していた WI-003 の統合テストは 00125 に変わる。
- 製造指示の年：新しい行はすべて、WI-003 の初期データと同じく実行日の工場の年を使う。一部は最大 108 日前に作成されたものであってもそうする。1〜4 月に初期データを実行すると、一部のデモ用製造指示は作成日より後の年を持つことになる。これは見た目だけの問題であり、デモ用製造指示に 2 つのカウンターを持たせる代わりに許容する。
- 環境：WI-003 の初期データと同様に、WI-003 の初期データが実行されたすべてのデータベース（統合テスト用データベースを含む）に入る。ダッシュボードの統合テストは、投入日付から離れた日付に固定した工場の時計のもとで自ら作成した製造指示に対して、または差分に対して検証しなければならず、投入済みの合計に対しては検証しない。方法は 003_DD で規定する。

これらの値は実行時に計算するので、初期データは 002_DB と同じ理由で生 SQL（migrationBuilder.Sql）で書く。

データベースロールと権限

変更なし。テーブル単位の権限付与は、後から追加した列にも及ぶ。

ロール	使用者	これらのテーブルへの権限	理由
pmai_app（実行時ログイン）	実行時の API	production_orders と products に SELECT、production_orders に UPDATE — 001_DB でテーブル単位の SELECT、INSERT、UPDATE を付与済み	ダッシュボードは読むだけで、画面 A の保存が新しい列を書く。DELETE と DDL は不要であり付与しない
オーナー（POSTGRES_USER）	マイグレーションのみ	オーナー — DDL	下の 3 つのマイグレーションはすべてオーナーとして実行する

API / DD 対応

DD 項目名は 003_BD \$3 の変数名。API 項目名は 003_DD-API で確定している。

テーブル列 / クエリ	DD 項目	API 項目
Q1 の status 別件数	statusCounts.* (項目 7～11)	statusCounts.total、.draft、.inProgress、.completed、.cancelled
Q3a の行 / total	overdue (項目 16、17)	overdue.total、overdue.orders[]
Q3b の行 / total	dueSoon (項目 18、19)	dueSoon.total、dueSoon.orders[]
production_orders.id、order_number、quantity、due_date、status、products.sku、name	一覧表示する製造指示の項目 (M-11、M-12、M-15)	orders[].id、.orderNumber、.quantity、.dueDate、.status、.product.sku、.product.name
Q2 の区分	workload (項目 20)	workload[] : { kind, weekStart?, orderCount, quantity }。kind は overdue、week、later のいずれか。10 エントリ
Q4 の行	topProducts (項目 22)	topProducts[] : { product.sku, product.name, openQuantity, activeOrderCount }
Q5 の週 / 月	completedThisWeek、completedThisMonth (項目 12、13)	completedThisWeek.{orderCount, quantity}、completedThisMonth.{...}
Q5 の window_on_time、window_count	onTime (項目 14)	onTime.onTimeCount、onTime.completedCount (率は表示用に導出し、重複して送らない)
Q5 の window_lead_days、window_count	leadTime (項目 15)	leadTime.averageDays (ない場合は null)、leadTime.orderCount
Q6 の行	completionTrend (項目 23)	completionTrend[] : { weekStart, orderCount }、12 エントリ
— (スナップショット時刻、工場の今日)	asOf (項目 4)	asOf (ISO 8601 UTC)、today (工場現地の日付)
production_orders.completed_at_utc	—	どのエンドポイントでも公開しない: 画面 A は表示せず (REQ-033)、ダッシュボードはその集計だけを公開する

性能見込み

002_DB と同じ規模の前提：年間約 15,000 件の製造指示。アクティブ集合（未完了の負荷）は数百から数千件台にとどまり、任意の 12 週間の完了は多くても約 3,500 件である。デモ用データベースは 124 件。

クエリ	見込み
Q1 ステータス別件数	テーブル全体の順次スキャンと 4 グループのハッシュ集約 — 10 万行あたり約 1 ミリ秒で、この件数なら何年も許容範囲
Q2、Q4	アクティブの部分インデックスのビットマップスキャンと、未完了の負荷に対する集約
Q3a、Q3b	アクティブの部分インデックスの順序付きスキャンで 10 行で停止。ウィンドウ関数の件数にはグループ全体が必要だが、それは未完了の負荷で上限がある
Q5、Q6	完了の部分インデックスの約 12 週間分の範囲スキャンと集約

したがってダッシュボード 1 回の読み込みは、少数の範囲限定スキャンと 1 回の全件数（Q1）である。テーブルが約 100 万行を超えた場合（Q1 を保持した件数に置き換える）、または未完了の負荷が履歴に対して小さくなった場合に見直す。

マイグレーションの影響と復旧限界

マイグレーション	種別	対象テーブル	稼働中のテーブルか	手順
AddProductionOrderCompletionTracking (2026-09-22)	追加、続いてデータ変換 (バックフィル)	production_orders (列、バックフィル、チェック制約 2 つ)	yes	expand/contract の expand ステップ。contract ステップは続かない
AddProductionOrderDashboardIndexes (2026-09-22)	追加	production_orders (部分インデックス 2 つ)	yes	単一ステップ。インデックスは CONCURRENTLY で作成
SeedDashboardDemoHistory (2026-09-22)	データ変換とデータ挿入 (保護付き)	production_orders 、 production_order_number_counters	yes	単一ステップ

3 つのマイグレーションを次の順に実行する：

1. AddProductionOrderCompletionTracking — 列、バックフィル、制約 (1 トランザクション) 。

- **マイグレーション種別:** 追加、続いてデータ変換 (バックフィル) — expand/contract 変更の expand ステップ (ai/rules/database.md) 。古いものを置き換えるわけではないので、後で contract するものはない。
- **手順:** ALTER TABLE production_orders ADD COLUMN completed_at_utc timestamptz NULL 。既定値のない NULL 許容列はカタログのみの変更で、テーブルの書き換えはない。次に UPDATE production_orders SET completed_at_utc = updated_at_utc WHERE status = 'Completed' (DEC-004) ：完了済みの製造指示では、最後の更新は完了の保存か、その後の備考・納期の編集なので、これが得られる最良の根拠であり、created_at_utc より前になることはない。その後で両方のチェック制約。
- **ロック:** 制約は同じトランザクション内で検証付きで追加し、ALTER が既に保持している ACCESS EXCLUSIVE ロックのもとでテーブルをスキャンする。上記の規模ではそのスキャンは 1 秒よりずっと短い。はるかに大きなテーブルなら、制約を NOT VALID で追加し、より弱いロックで済む別のマイグレーションで検証する。
- **デプロイ順序:** 移行後のスキーマに対して誰かが製造指示を完了できるようになる前に、completed_at_utc を設定するバックエンドのバージョンが動いていなければならない。マイグレーション 1 の後に古いバックエンドが製造指示を完了すると ck_..._matches_status に違反し、その保存は数値を黙って壊すのではなくサーバーエラーで失敗する。既存の Compose の手順は、バックエンドを停止したままマイグレーションを実行し、その後で新しいイメージを起動するので (deploy/README.md) 、そこにはその隙間はない。
- **副作用:** バックフィルは完了済みの各行を更新するので、その xmin が変わる。したがって、完了済みの製造指示で既に開いている画面 A の編集フォームは、保存時に通常の古い保存の競合 (MSG-E009) になり、再読み込みが必要になる。これは無害で、完了済みの製造指示にだけ当てはまる。
- **データ復旧限界:** 戻すと列が削除されるので、マイグレーション後に記録された完了日時はすべて失われる。バックフィルと同じく updated_at_utc から近似的にしか再構成できない。
- **ロールバック計画:** 両方の制約を削除し、続いて列を削除する。その後、古いバックエンドはそのまま動く。

2. AddProductionOrderDashboardIndexes — 2 つの部分インデックス。

- **マイグレーション種別:** 追加。それぞれ CREATE INDEX CONCURRENTLY IF NOT EXISTS を migrationBuilder.Sql(..., suppressTransaction: true) で実行する。002_DB のインデックスのマイグレーションとまったく同じ。
- **アトミックでない:** 途中で失敗すると INVALID なインデックスが残り、再試行の前に削除 (DROP INDEX CONCURRENTLY) しなければならない。SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid; で確認する。
- **データ復旧限界:** なし — データは書き込まない。

- **ロールバック計画:** 両方に `DROP INDEX CONCURRENTLY IF EXISTS`。戻してもダッシュボードが遅くなるだけで、誤りにはならない。

3. **SeedDashboardDemoHistory** — デモ用の履歴（1 トランザクション）。

- **マイグレーション種別:** データ変換とデータ挿入（保護付き。上の「保護」を参照）。最大 16 の投入行の日付を変更し、最大 44 行を挿入し、投入年のカウンターを進める。
- **データ復旧限界:** 日付を変更した行の元の `created_at_utc`、`updated_at_utc`、`completed_at_utc` の値 — それ自体、以前の実行日に対する相対値のデモ用の値 — は上書きされ、戻しても復元されない。実際の製造指示には触れない。
- **ロールバック計画:** `DELETE FROM production_orders WHERE id::text LIKE '0197e4a0-0000-7000-8002-%'` で挿入した 44 行を削除する。カウンターは進めたままにし、製造指示番号が再利用されないようにする。日付を変更した 16 件は新しいタイムスタンプのまま残る。
- **画面 B への影響:** デモ用データベースは 80 件から 124 件に増える。画面 B に新しい動作はないが、投入済みの数値を記載している WI-003 の文書とテスト — 002_DB の初期データの表、80 行の合計・ステータスの分布・次の番号 `00081` を検証するテスト — は、計画改訂 2 で新しい数値に更新する。相対日付を検証するテストは、投入済みのアクティブな製造指示の納期が変わらないので影響を受けない。

未決事項

決定事項	選択肢	推奨	状態
完了の記録：列か履歴テーブルか	列／履歴テーブル	列	決定済み — <code>work-items/WI-004/decisions.md</code> DEC-003
既存の完了済み製造指示のバックフィル	初期データ + <code>updated_at_utc</code> からのバックフィル／NULL のまま	初期データ + バックフィル	決定済み — DEC-004
デモ用の履歴：製造指示を追加するか、既存の日付を変更するか、変換するか	約 40 件追加／16 件の日付変更／一部を変換／追加と日付変更の両方	両方	決定済み — DEC-013（ユーザー）
初期データで 8 週間の見通しを超える納期のアクティブな製造指示	4 件追加／なし	4 件追加	決定済み — DEC-014（Claude。003_DB のレビューでのユーザーの異議を受け付ける）
ダッシュボードが一貫した 1 つのスナップショットを得る方法	<code>REPEATABLE READ READ ONLY</code> トランザクション／1 つの CTE の文／保証なし	トランザクション	決定済み — DEC-015
完了のルールをデータベースで強制するか	2 つの CHECK 制約／アプリケーションのみ	制約	決定済み — ここに記録（どちらのルールも時計に関係なく成り立つ。制約を参照）
ステータス別件数のインデックス	なし／ステータスのインデックス／件数の保持	約 100 万行まではなし	決定済み — ここに記録